Exercise Sheet 06

Monday Morning

House rules.

- One file per exercise, in `my_work/`: `ex1_traceback.py`, `ex2_try_except.py`, …
- Run everything **from the lesson folder**. The broken file is `data/brightwell_payroll_broken.csv`; leave it broken.
- Today, errors are the point. When something crashes, read the **last line** first (what), then the **line number** (where), then decide: fix the code, or catch the error on purpose?
- Hand in: commit and push to `lesson-06`, open a pull request, and tell Lars which exception type you met most often.

Exercise 1 Read the traceback

8 min · `ex1_traceback.py`

> **From Mo (IT):** *"Before you catch errors, learn to read them. Python tells you exactly what went wrong. People just don't look."*

GIVEN (TYPE IT EXACTLY, INCLUDING THE MISTAKE)

```
employees = ["Priya", "Mo", "Siobhan"]
salaries = [32000, 41000, 38000]
for i in range(4):
    print(employees[i], salaries[i])
```

(a) Run it. It crashes. Write the **line number** and the **error type** as a comment in the file. Then fix the loop so it works for any number of employees.

(b) Add `print(salary)` at the bottom (singular, on purpose). Run it, write down the error type, then replace it with a line that prints the total of all salaries.

EXPECTED OUTPUT

```
Priya 32000
Mo 41000
Siobhan 38000
Total: 111000
```

Hint: A traceback reads bottom-up: the last line is the error, the line above it points at your code. The error types you'll meet today: `IndexError`, `NameError`, `ValueError`, `TypeError`, `ZeroDivisionError`, `KeyError`.

Exercise 2 Catch it

10 min · `ex2_try_except.py`

> **From Priya (Payroll):** *"The export has 'n/a' in the salary column for new starters. Your lovely script from last month falls over on row 2. Can it just… carry on?"*

GIVEN

```
raw_values = ["£32,500", "n/a", "", "41000", "thirty", "£1,250.50"]
```

(a) Loop over the values. Remove £ and commas, then try to turn the text into a float **inside a `try` block**.

(b) If it works, print the original text (with `!r`, left-aligned in 12) and the number. If it raises a `ValueError`, print "could not read" instead.

(c) Count the good and the bad ones and print the totals.

EXPECTED OUTPUT

```
'£32,500'    -> 32500.0
'n/a'        -> could not read
''           -> could not read
'41000'      -> 41000.0
'thirty'     -> could not read
'£1,250.50'  -> 1250.5
Read 3, skipped 3
```

Hint: `f"{text!r:<12}"` shows the quotes and pads to 12. Put the "it worked" print in an `else:` after the `except:`, so only the risky line sits inside `try`.

Exercise 3 Which error is it?

10 min · `ex3_which_error.py`

> **From Siobhan (Reward):** *"'Something went wrong' is not a reason. Zero hours is a different problem from a missing salary. Tell me which."*

GIVEN

```
pairs = [(32500, 37.5), (41000, 0), ("n/a", 37.5), (29000, None), (24950, "22.5")]
```

(a) For each pair, compute the hourly rate: `float(salary) / 52 / float(hours)`, inside a `try`.

(b) Catch `ZeroDivisionError`, `ValueError` and `TypeError` in three separate `except` blocks, each with its own message.

(c) In the `else` block, print the rate with 2 decimals.

EXPECTED OUTPUT

```
32500, 37.5: £16.67 per hour
41000, 0: zero hours, is this person on leave?
n/a, 37.5: that's not a number
29000, None: a value is missing
24950, 22.5: £21.32 per hour
```

Hint: Before you run it, write next to each pair which error you expect. `float("22.5")` is fine; `float(None)` is not.

Exercise 4 Raise your own

10 min · `ex4_raise.py`

> **From Siobhan:** *"Grades go from 1 to 6. Somebody entered a 7 and MACRO-9000 gave them a 40% bonus. I want the script to refuse, loudly."*

GIVEN

```
grades = [3, 7, 0, "4", 5]
```

(a) Write `check_grade(grade)`. If the grade is not an `int`, `raise TypeError` with a message that includes the value (use `!r`). If it's outside 1–6, `raise ValueError` with a message that includes the number. Otherwise return it.

(b) Loop over the grades and call it inside `try`. Print "Rejected:" and the message for each error, "OK:" and the grade otherwise.

(c) Add a `finally:` block that counts how many grades were checked, and print the count at the end.

EXPECTED OUTPUT

```
OK: 3
Rejected: grade must be between 1 and 6, got 7
```

```
Rejected: grade must be between 1 and 6, got 0
Rejected: grade must be a whole number, got '4'
OK: 5
Checked 5 grades
```

Hint: `except ValueError as e:` puts the message in `e`. `isinstance(grade, int)` asks "is this an int?". `finally` runs every time, error or not, which is exactly what a counter wants.

Exercise 5 The broken file

12 min · `ex5_broken_file.py`

> **From Priya:** *"`data/brightwell_payroll_broken.csv` is this month's export. Two hundred rows, and I know at least four are wrong. Find them, skip them, and give me the average hourly rate of everyone else."*

GIVEN

```
import csv
```

(a) Read the file with `csv.DictReader`. For each row, inside one `try`: compute the hourly rate (salary / 52 / hours) and the total pay (salary + bonus), converting with `float()`.

(b) Catch `ValueError`, `TypeError` and `ZeroDivisionError` together in one `except` as `e`. Store the employee ID and `type(e).__name__` for each bad row; store the hourly rate for each good one.

(c) Print the counts, the bad rows with their error type, and the average hourly rate of the good rows.

EXPECTED OUTPUT

```
Good rows: 194
Bad rows: 6
  EMP1178  ValueError
  EMP1037  ValueError
  EMP1081  ZeroDivisionError
  EMP1065  TypeError
  EMP1150  ValueError
  EMP1120  ValueError
Average hourly rate: £17.08
```

Hint: Catch several types at once with a tuple: `except (ValueError, TypeError, ZeroDivisionError) as e:`. One row is missing its last two columns, so `row["bonus"]` is `None`: that's the TypeError.

Exercise 6 Print debugging

8 min · `ex6_print_debugging.py`

> **From Gerald (MD):** *"The bonus pool script says £0. Siobhan says that can't be right. I would like it to be right, but I'd settle for knowing why it isn't."*

GIVEN (TYPE IT EXACTLY; IT HAS A LOGIC BUG, NO CRASH)

```
bonuses = {"Priya": 1200, "Mo": 950, "Siobhan": 1500, "Gerald": 0}
for name, bonus in bonuses.items():
    pool = 0
    if bonus > 0:
        pool += bonus
print("Bonus pool: £" + f"{pool:,}")
```

(a) Run it. Then add a `print(name, bonus, pool)` inside the loop, run again, and *watch* the pool. Where does the money go?

(b) Fix the bug. Remove your debugging print. Add a second line printing how many people got a bonus.

EXPECTED OUTPUT

```
Bonus pool: £3,650
People with a bonus: 3
```

Hint: Printing a variable inside a loop is the oldest debugging tool there is, and it still works. VS Code's debugger (lecture) does the same thing without editing the file: put a breakpoint on `pool += bonus`, press F5, step with F10.

Exercise 7 A safety net STRETCH

12 min · `ex7_safe_float.py`

> **From Mo:** *"You'll write 'try float, except give up' a hundred more times. Write it once, as a function, and make it log what it gave up on."*

GIVEN

```
import csv
```

(a) Write `safe_float(text, default=None)`: returns `float(text)`, or the default if that raises `ValueError` or `TypeError`.

(b) Read the broken file. For every row and each of the columns `annual_salary`, `weekly_hours`, `bonus`: if `safe_float` gives `None`, count it per column in a dict and add a line "EMPxxxx: column = 'value'" to a list.

(c) Print the counts per column, write the lines to `my_work/bad_rows.txt`, print how many you logged, then print the file.

EXPECTED OUTPUT

```
annual_salary: 2 unreadable
bonus: 2 unreadable
weekly_hours: 1 unreadable
Logged 5 problems to bad_rows.txt
EMP1178: annual_salary = 'n/a'
EMP1037: annual_salary = ''
EMP1065: bonus = None
EMP1150: bonus = 'tbc'
EMP1120: weekly_hours = 'full'
```

Hint: Zero hours is a valid float, so it doesn't appear here. That's the difference between "unreadable" and "wrong": a safety net catches the first kind. The second kind needs a human, or a `raise`.

Bug hunt Dave's Leftovers #6

15 min · `bughunt/robust_bug.py`

Dave's "robust" payroll total, with error handling he added in 2019. There are **three** bugs: one is a SyntaxError in the error handling itself (Python 2 habits), one double-counts money without anyone noticing, and one is off by exactly one.

EXPECTED OUTPUT ONCE FIXED

```
Total salary: £6,322,500
Skipped 3 rows: EMP1178, EMP1037, EMP1120
```

Commit each fix separately. For the double-counting bug, explain in the commit message *which* row's salary was counted twice.